

AI Assistant Coding

Name: Vishnu Vardhan

HTNO: 2303A510A0

Batch-14

Date: 12-03-2026

Assignment - 13.4

Task 1: Refactoring Data Transformation Logic

AI Prompt

"I have a Python loop that doubles values in a list. How can I refactor this to be more 'Pythonic' and concise while maintaining the same output?"

Refactored Code

```
values = [2, 4, 6, 8, 10]
```

```
# Refactored using List Comprehension
```

```
doubled = [v * 2 for v in values]
```

```
print(doubled)
```

Code Explanation

Legacy Issue: The original code used an explicit for loop and the `.append()` method, which is more verbose and slightly slower in Python.

Refactoring: I applied a list comprehension. This is the standard Pythonic way to transform data. It is more readable and executes faster because it is optimized at the interpreter level.

Output: [4, 8, 12, 16, 20]

Task 2: Improving Text Processing Code Readability

AI Prompt

"I am joining a list of strings into a sentence using a loop and concatenation. Is there a more efficient way to do this in Python?"

Refactored Code

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
```

```
# Refactored using the .join() method
```

```
message = " ".join(words)
```

```
print(message)
```

Code Explanation

Legacy Issue: Using `+=` in a loop to build strings is inefficient. In Python, strings are immutable, so every `+=` creates a brand-new string object in memory.

Refactoring: I used the `.join()` method. It calculates the memory needed once and performs the operation in a single pass, which is significantly more memory-efficient.

Output: Refactoring with AI improves quality

Task 3: Safer Access to Configuration Data

AI Prompt

"Refactor this if-else block that checks for a dictionary key. I want a cleaner way to handle missing keys with a default value."

Refactored Code

Python

```
config = {"host": "localhost", "port": 8080}
```

```
# Refactored using the .get() method
```

```
print(config.get("timeout", "Default timeout used"))
```

Code Explanation

Legacy Issue: Manually checking for a key using an if statement adds unnecessary lines of code (boilerplate).

Refactoring: I used the `.get()` method. This method allows you to attempt to access a key and provide a "fallback" value if the key does not exist, all in one line.

Output: Default timeout used

Task 4: Refactoring Conditional Logic for Scalability

AI Prompt

"I have a long if-elif chain for mathematical operations. How can I refactor this to be more scalable and avoid multiple conditional checks?"

Refactored Code

```
import operator
```

```
action = "divide"
```

```
x, y = 10, 2
```

```
# Refactored using a Dispatch Dictionary (Mapping)
```

```
operations = {
```

```
    "add": x + y,
```

```
    "subtract": x - y,
```

```
    "multiply": x * y,
```

```
    "divide": x / y if y != 0 else None
```

```
}
```

```
result = operations.get(action, None)
```

```
print(result)
```

Code Explanation

Legacy Issue: Using if-elif for many operations becomes hard to read and maintain as the list grows.

Refactoring: I implemented a Dictionary Mapping (also known as a Dispatch Table). This treats the "action" as a key to look up the result directly. It is much easier to add a new operation (like "power") by just adding one line to the dictionary.

Output: 5.0

Task 5: Simplifying Search Logic in Collections

AI Prompt

"How can I check if an item exists in a list without using a manual for-loop and a boolean flag?"

Refactored Code

```
inventory = ["pen", "notebook", "eraser", "marker"]

# Refactored using the 'in' operator
found = "eraser" in inventory

print("Item Available" if found else "Item Not Available")
```

Code Explanation

Legacy Issue: The manual loop and break statement are unnecessary for a simple membership check.

Refactoring: I used the in operator. It is the most readable and efficient way to check for existence in a collection. It replaces five lines of code with one.

Output: Item Available